

Lecture 13 – Selection, Bubble & Insertion Sort

DSA MASTERY SERIES | LeetCode 912 | C++ Focused | Print-Ready A4

PROBLEM STATEMENT

LeetCode 912 – Sort an Array

Ek integer array `nums` diya gaya hai. Task hai array ko **ascending order** me sort karna aur sorted array return karna.

EXAMPLES

Example 1: Input: [5, 2, 3, 1] → Output: [1, 2, 3, 5]

Example 2: Input: [5, 1, 1, 2, 0, 0] → Output: [0, 0, 1, 1, 2, 5]

Example 3: Input: [3, 2, 1] → Output: [1, 2, 3]



1. Selection Sort

INTUITION & REAL-LIFE ANALOGY

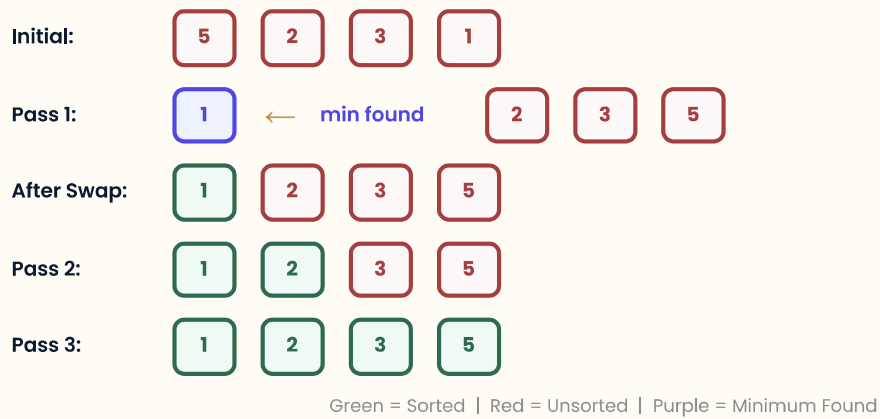
Analogy: Socho tumhe ek line me khade 10 students me se sabse chhote (shortest) student ko line ke start me bhejna hai. Phir bache hue 9 me se sabse chhote ko second position par, aur aise hi continue karna hai.

Har baar **unsorted part se minimum element** dhundh kar usse current position par place karte hain. Ye **"select and place"** strategy hai — isliye naam *Selection Sort* hai.

ALGORITHM WORKFLOW

- Array ko do parts me divide karo: **Sorted Part** (left side) aur **Unsorted Part** (right side).
- Current index `i` ko minimum maan lo.
- Unsorted part me actual minimum element find karo (index `minIndex`).
- Minimum element ko current index `i` ke saath **swap** kar do.
- Step 2-4 har position `i = 0` se `n-2` tak repeat karo.

Visual: Selection Sort Concept



C++ Implementation

```
// Selection Sort - LeetCode 912
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        // Traverse complete array
        for (int i = 0; i < n - 1; i++) {

            // Assume current index has minimum element
            int minIndex = i;

            // Find actual minimum in unsorted part
            for (int j = i + 1; j < n; j++) {
                if (nums[j] < nums[minIndex]) {
                    minIndex = j;
                }
            }

            // Place minimum at its correct position
            swap(nums[i], nums[minIndex]);
        }

        return nums;
    }
};
```

STEP-BY-STEP DRY RUN

Input: nums = [5, 2, 3, 1]

Pass 1 (i = 0): minIndex starts at 0 (value 5). j=1 → 2<5, minIndex=1. j=2 → 3<2? No. j=3 → 1<2, minIndex=3.

Swap nums[0]↔nums[3]: [1, 2, 3, 5]

Pass 2 (i = 1): minIndex=1 (value 2). j=2 → 3<2? No. j=3 → 5<2? No. **No swap needed.** Array: [1, 2, 3, 5]

Pass 3 (i = 2): minIndex=2 (value 3). j=3 → 5<3? No. **No swap needed.** Array: [1, 2, 3, 5]

Final Output: [1, 2, 3, 5]

COMPLEXITY ANALYSIS

Case	Time Complexity	Reason
Best Case	$O(n^2)$	Har baar nested loop chalega, chahe array sorted ho ya na ho
Average Case	$O(n^2)$	Approximately $n^2/2$ comparisons hote hain
Worst Case	$O(n^2)$	Reverse sorted array me bhi same comparisons

Space Complexity: $O(1)$ — In-place sorting, extra memory nahi lagti.

EDGE CASES & GOTCHAS

- **Empty array / Single element:** Loop nahi chalega, directly return hoga. Handle ho jata hai.
- **Duplicate elements:** Selection Sort **stable nahi hai** — equal elements ka relative order change ho sakta hai swap ke wajah se.
- **Already sorted array:** Phir bhi $O(n^2)$ comparisons honge — adaptive nahi hai.

INTERVIEW TIP

Interviewer pooch sakta hai: "*Selection Sort ko stable kaise banaoge?*" — Answer: Minimum element ko swap karne ke bajaye, **shift karke insert** karo. Lekin phir wo Insertion Sort ban jayega!



2. Bubble Sort

INTUITION & REAL-LIFE ANALOGY

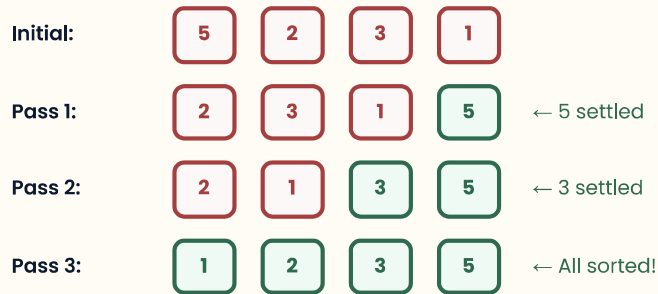
Analogy: Socho ek fish tank me bubbles hain — halki (light) bubbles paani ke upar ki taraf udti hain. Similarly, Bubble Sort me **chhote elements left ki taraf "bubble up" hote hain** aur bade elements right side settle ho jate hain.

Har pass me adjacent elements compare karte hain. Agar left wala bada hai, to swap kar do. Har complete pass ke baad, **sabse bada element apni correct position par pahunch jata hai** — jaise bubble upar jaati hai.

ALGORITHM WORKFLOW

1. Outer loop: $i = 0$ se $n-2$ tak — total $n-1$ passes.
2. Har pass me inner loop: $j = 0$ se $n-i-2$ tak — last i elements already sorted hain.
3. $nums[j]$ aur $nums[j+1]$ ko compare karo.
4. Agar $nums[j] > nums[j+1]$, to dono ko **swap** kar do.
5. Har pass ke end me, largest element apni final position par hota hai.

Visual: Bubble Sort - Bubbles Rising to Surface



Green = Sorted/Settled | Red = Still Bubbling

C++ Implementation

```
// Bubble Sort - LeetCode 912
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        // Total n - 1 passes
        for (int i = 0; i < n - 1; i++) {

            // Compare adjacent elements
            // Last i elements already sorted hain
            for (int j = 0; j < n - i - 1; j++) {

                // Swap if wrong order me hain
                if (nums[j] > nums[j + 1]) {
                    swap(nums[j], nums[j + 1]);
                }
            }
        }

        return nums;
    }
};
```

STEP-BY-STEP DRY RUN

Input: nums = [5, 2, 3, 1]

Pass 1: [5,2,3,1] → Swap(5,2) → [2,5,3,1] → Swap(5,3) → [2,3,5,1] → Swap(5,1) → **[2,3,1,5]** (5 settled at end)

Pass 2: [2,3,1,5] → 2<3? Yes, no swap → 3>1? Yes, Swap(3,1) → **[2,1,3,5]** (3 settled)

Pass 3: [2,1,3,5] → 2>1? Yes, Swap(2,1) → **[1,2,3,5]** (2 settled)

Final Output: [1, 2, 3, 5]

COMPLEXITY ANALYSIS

Case	Time Complexity	Reason
Best Case	$O(n^2)$	Basic implementation me har baar nested loop chalega
Average Case	$O(n^2)$	Approximately $n^2/2$ comparisons and swaps
Worst Case	$O(n^2)$	Reverse sorted array me maximum swaps

Space Complexity: $O(1)$ — In-place sorting.

Optimization Tip: Agar kisi pass me **zero swaps** hue, matlab array sorted hai. Ek swapped flag add karke Best Case $O(n)$ banaya ja sakta hai!

EDGE CASES & GOTCHAS

- **Optimization opportunity:** swapped flag se early termination possible — already sorted array me $O(n)$ ho jayega.
- **Stable Sort:** Haan, Bubble Sort **stable** hai — equal elements ka relative order change nahi hota.
- **Adaptive:** Optimized version me partially sorted data pe better performance deta hai.

INTERVIEW TIP

Interviewer aksar poochta hai: *"Bubble Sort ko optimize kaise karoge?"* — swapped flag lagao. Agar koi pass me swap nahi hua, loop break kar do. Isse Best Case $O(n)$ ho jata hai. Ye small detail badi difference create karti hai!



3. Insertion Sort

INTUITION & REAL-LIFE ANALOGY

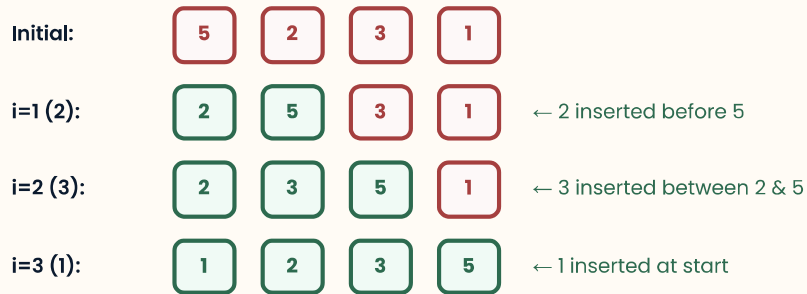
Analogy: Socho tumhe playing cards sort karna hai. Pehla card already sorted maan lo. Dusra card uthao aur pehle ke saath compare karke sahi jagah pe rakh do. Teesra card uthao aur already sorted cards me uski correct position dhundh ke insert kar do.

Insertion Sort bhi exactly aise hi kaam karta hai — **har iteration me current element ko already sorted part me uski correct position par insert karte hain**. Left side hamesha sorted rehta hai, right side unsorted.

ALGORITHM WORKFLOW

1. Pehla element ($i = 0$) ko already sorted maan lo.
2. Current element $nums[i]$ ko uthao ($i = 1$ se start).
3. Sorted part (left side) me backward move karo: $j = i$ se 1 tak.
4. Agar $nums[j] < nums[j-1]$, to swap karte raho.
5. Jab $nums[j] \geq nums[j-1]$ mile, correct position mil gayi — break kar do.
6. Step 2-5 har element ke liye repeat karo.

Visual: Insertion Sort - Like Sorting Playing Cards



Green = Sorted Hand | Red = Cards Yet to Pick

C++ Implementation

```
// Insertion Sort - LeetCode 912
class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        // Assume first element already sorted
        for (int i = 1; i < n; i++) {

            // Insert current element into sorted part
            for (int j = i; j > 0; j--) {

                // Move current element left until correct position
                if (nums[j] < nums[j - 1]) {
                    swap(nums[j], nums[j - 1]);
                }
                else {
                    // Correct position found, stop shifting
                    break;
                }
            }
        }

        return nums;
    }
};
```

STEP-BY-STEP DRY RUN

Input: nums = [5, 2, 3, 1]

Initial: [5 | 2, 3, 1] — 5 sorted maana, baaki unsorted

i=1 (element 2): 2 < 5? Yes → Swap → **[2, 5 | 3, 1]**

i=2 (element 3): 3 < 5? Yes → Swap → [2, 3, 5 | 1] → 3 < 2? No → Break → **[2, 3, 5 | 1]**

i=3 (element 1): 1 < 5? Yes → Swap → [2, 3, 1, 5] → 1 < 3? Yes → Swap → [2, 1, 3, 5] → 1 < 2? Yes → Swap → **[1, 2, 3, 5]**

Final Output: [1, 2, 3, 5]

COMPLEXITY ANALYSIS

Case	Time Complexity	Reason
Best Case	$O(n)$	Already sorted array — har element ek hi comparison, break ho jata hai
Average Case	$O(n^2)$	Random data me approximately $n^2/4$ shifts hote hain
Worst Case	$O(n^2)$	Reverse sorted array me har element ko start tak shift karna padta hai

Space Complexity: $O(1)$ — In-place sorting, sirf variables use hote hain.

EDGE CASES & GOTCHAS

- **Best performer for small arrays:** Insertion Sort small data pe sabse fast hota hai in practice — isliye hybrid algorithms (Timsort, Introsort) isse use karte hain.
- **Stable Sort:** Haan, equal elements ka order preserve hota hai — **stable** hai.
- **Adaptive:** Haan, partially sorted data pe bahut accha performance deta hai.
- **Online algorithm:** Data stream me elements aate ja rahe hain — Insertion Sort real-time sorting ke liye best hai.

INTERVIEW TIP

Interviewer pooch sakta hai: *"Insertion Sort ko optimize kaise karoge swaps kam karke?"* — Answer: Swap karne ke bajaye, **current element ko store karo** aur sorted elements ko right shift karo. Jab correct position mile, tab directly insert karo. Isse assignments kam hote hain!



4. Algorithm Comparison

HEAD-TO-HEAD COMPARISON

Property	Selection Sort	Bubble Sort	Insertion Sort
Time (Best)	$O(n^2)$	$O(n^2)$ ($O(n)$ optimized)	$O(n)$
Time (Worst)	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(1)$	$O(1)$	$O(1)$
Stable?	No	Yes	Yes
In-Place?	Yes	Yes	Yes
Adaptive?	No	Partially	Yes
Swaps	$O(n)$	$O(n^2)$	$O(n^2)$
Real-Life Use	Memory write kam karna ho	Teaching purposes	Small / nearly sorted data

PATTERN TRANSFER & RELATED PROBLEMS

- **LeetCode 912** — Sort an Array (Base problem)
- **LeetCode 148** — Sort List (Linked List pe sorting)
- **LeetCode 75** — Sort Colors (Dutch National Flag — counting sort pattern)
- **LeetCode 147** — Insertion Sort List (Insertion Sort on Linked List)
- **LeetCode 215** — Kth Largest Element (Selection Sort concept se kth element nikaal sakte hain)

KEY TAKEAWAYS

- **Selection Sort:** Minimum dhundho aur place karo. Simple but slow. **Swaps kam** karta hai — $O(n)$ swaps only.
- **Bubble Sort:** Adjacent compare karke bubble up karo. **Stable** hai. Optimized version me Best Case $O(n)$.
- **Insertion Sort:** Cards jaisa sort karo. **Best for small / nearly sorted data.** Stable + Adaptive + Online.
- **Sabhi $O(n^2)$ hain** — large data ke liye Merge Sort, Quick Sort, ya Heap Sort use karo.
- **Interview me:** In algorithms ka intuition + complexity + stability + adaptiveness yaad rakhna must hai.



QUICK REVISION CHEATSHEET

Selection Sort

Strategy: Min find karo → Swap karo

Time: $O(n^2)$ all cases

Space: $O(1)$ | Stable: **No**

Analogy: Sabse chhote student ko front pe bhejna

Bubble Sort

Strategy: Adjacent compare → Swap → Bubble up

Time: $O(n^2)$ | Best(opt): $O(n)$

Space: $O(1)$ | Stable: **Yes**

Analogy: Halki bubbles paani ke upar udti hain

Insertion Sort

Strategy: Current element ko sorted part me insert karo

Time: Best $O(n)$ | Worst $O(n^2)$

Space: $O(1)$ | Stable: **Yes** | Adaptive: **Yes**

Analogy: Playing cards ko haath me sort karna

Common Mistakes

- × Bubble Sort ko $O(n)$ best case batana bhoool jana
- × Selection Sort ko stable batana (galat!)
- × Inner loop bounds galat likhna
- × Already sorted array pe complexity confuse hona

Must Remember Code

Selection: `for(j=i+1; j<n; j++)`

Bubble: `for(j=0; j<n-i-1; j++)`

Insertion: `for(j=i; j>0; j--)`

Loop bounds hi sabse important hai!

Interview Questions

Q1: Kaunsa sort stable hai?

→ Bubble & Insertion: **Yes** | Selection: **No**

Q2: Nearly sorted array pe best kaunsa?

→ **Insertion Sort** — $O(n)$ time

Q3: Minimum swaps kisme hote hain?

→ Selection Sort — exactly $n-1$ swaps

Harshal Chauhan

MADE BY HARSHAL CHAUHAN